

# Yanjin Li

自学 AI 工程与研究 · 兼具产品与金融背景

我是一个靠自学成长的建造者 —— 高 agency、强好奇心，以及不断尝试的勇气。我的路径从自学法语与日语，到在 **ESCP** 攻读管理学与金融硕士（巴黎与伦敦）；从 **TikTok** 策略团队、**滴滴** 产品运营，到 **汇丰** 全球市场部与 **法国巴黎银行** 公司金融实习 —— 横跨北京、伦敦与巴黎。

自 2026 年初起，我完全靠阅读论文与网络课程自学 AI，随后在三个月内做了 **11 个项目**，让每一个想法变得真实 —— 从训练方法研究（一个无反向传播训练的 **4.16B 参数模型**）、schema-first 的跨设备 AI 系统（**42 份 JSON Schema**），到可信应用 ML 与已上线的本地优先产品。我想在这里啃更难的问题，和真正优秀的人一起。

geraldineliyanjin@gmail.com    github.com/Yanjin-ai    yanjin-ai.github.io/portfolio    画作 · @yanjin.l.painting

## 目录 — 11 个项目

按类型分组，点击任意条目跳转到对应项目的完整说明。

### A · 研究与训练方法

以缩略技术报告呈现：目标 → 方法 → 分阶段推进 → 结论 → 教训。

- 01

**Zero Gradient**    无反向传播训练的 4.16B MoE 语言模型；能力边界研究。

research
- 02

**NeuroGolf (ARC-AGI)**    把 400 个 ARC 任务编译成最小 ONNX 电路；逆向评分器、抗过拟合。

research
- 03

**Nemotron Post-training**    30B MoE 推理模型后训练流水线；失败的完整归因。

research
- 04

**Orbit Wars**    多智能体博弈程序；评估方法论研究（离线 vs 线上）。

research

### B · AI 系统与架构

以架构呈现：数据模型、schema、代码库、核心机制、设计决策。

- 05

**Gemma4all**    跨设备任务系统；schema-first、事件溯源、本地优先、人在回路审批。

system
- 06

**VoicelInput**    macOS 按键语音输入；纯 Swift，零依赖。

system

C · 应用 ML 与决策系统

以 ML 系统呈现：数据、模型、校准/不确定性/公平性，以及运营决策层。

- 07

**TriageGeist** 急诊分诊决策栈；标签泄漏发现、conformal 不确定性、公平性审计。

applied-ml
- 08

**Maze Crawler** 分层多单位策略 agent；BFS 规划、冲突解算、CI。

applied-ml

D · AI 产品与工具

以产品呈现：定位、功能、如何构建、当前状态。

- 09

**AI Showrunner** 一句话 → 带字幕竖屏短剧；VL 评审闭环、角色一致性。

product
- 10

**Offline French Companion** 纯浏览器 LLM 法语学习助手；词典 + 语音 + OCR + 朗读，完全离线。

product
- 11

**Atelier Post** 已上线的艺术作品发布工具；一套代码覆盖 Web、PWA、iOS。

product

A · 研究与训练方法

在单块 16 GB T4 上，**全局禁用反向传播**训练了一个 4.16B 参数的内容路由 MoE 语言模型，随后用五阶段实验精确测绘这种模型能学会什么、学不会什么，并用对照实验证明限制因素在架构，而非缺少梯度。

**SCALE** 4.16B params, 0.42% experts updated/step    **RESULT** test ppl 1355 (vs 159 same-budget backprop control)

**ARTIFACTS** paper draft · 8-pg workshop paper · arXiv package

|      |                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 目标   | 检验层级本地学习规则（无端到端梯度）能否在单卡显存硬约束下扩展到数十亿参数的语言模型，并精确刻画由此产生的能力天花板。                                                                                                                                                                                                                                                                                                                           |
| 方法   | 用层级本地损失 + 深度监督读出头替代梯度；EMA 原型内容路由 + 容量控制；冻结因果注意力（reservoir computing 思路）。4 层 × 950 专家，top-2 路由，BPE-32k，WikiText-103。完全确定性可复现。用同预算反向传播 Transformer（42M）作为代价对照。                                                                                                                                                                                                                          |
| 推进过程 | <p><b>A–D</b> 构建与扩放 —— nano 原型 → T4 上 4B。找到有效三角组合：极稀疏更新预算（每步 0.42%）+ 内容路由 + 深度监督。困惑度 1355；量化出相对 BP 对照（159）的 3–9 倍代价。</p> <p><b>E</b> 寻找边界 —— 在三类任务上后训练：情感 79%，NLI 随机，算术随机。一条清晰的、由任务结构决定的边界。</p> <p><b>F–G</b> 模型内修复 —— 5 条改进路线（数据/目标/注意力/非塌缩读出/深层 BP），无一突破边界。三个机制假说被探针证伪。</p> <p><b>H</b> 对照研究 —— 隔离的标准可训练注意力 Transformer 瞬间解决同样的合成任务（100%），真实 SNLI 达 69.97%（对比 33.4%）。瓶颈定位到冻结架构。</p> |
| 结论   | 能力天花板在架构而非算法：冻结注意力无法表示关系结构，与梯度预算无关。"能力边界作为任务结构的函数"（词袋组合性 < 关系对齐 < 多步深度）是一条与 scaling laws 正交的研究轴。                                                                                                                                                                                                                                                                                      |
| 教训   | 当作真正的研究计划来跑：预注册判定门、假说→探针→证伪循环、用 LOCKED/INTERPRETATION/PROPOSAL 标签区分事实与推断。发现并公开纠正了两个由训练工件造成的假性结果，此后所有能力论断统一改用与任务无关的闭式探针。                                                                                                                                                                                                                                                               |
| 局限   | 1355 的困惑度离可用语言模型很远（仅在词袋类任务与语言建模上有效）；对照模型仅 12–21M，其 100% 迁移不能直接外推到 4B（论文已说明）；即便对照架构也在 k≥4 多步推理撞墙，生成式 GSM8K 仅 2%。                                                                                                                                                                                                                                                                       |

# 02 NeuroGolf — ARC-AGI Circuit Compiler RESEARCH

Python · ONNX / onnxruntime · 2026-07

把一个 ARC-AGI"神经高尔夫"比赛 —— 为 400 个 ARC 任务各自最小化一个能复现其变换的静态 ONNX 网络，评分  $\max(1, 25 - \ln(\text{params} + \text{memory}))$  —— 从一个学习问题重构为**程序合成 + 电路 golfing**：把评分器逆向成成本模型，再把已知的任务规则编译成最小的、抗过拟合的整数权重电路。

**SCOPE** 400 ARC tasks · one ONNX circuit each    **BASELINE** strongest public bundle LB 7243 (3-mo community-tuned)  
**DELIVERABLE** transferable cost-model + methodology note

**问题重构**      比赛表面是 few-shot 学习，但官方每题一个 Python 生成器 (arc-gen) 就是真值规则的可执行形式 —— 同时也是私有集的同分布采样器；符号 DSL solver (arc-dsl) 覆盖 280 题。于是任务不是"推断规则"，而是"把已知规则编译成满足约束、抗过拟合的最小 ONNX 电路"。识别这一点是最大的杠杆。

**方法**      搭建工具链：复刻官方评分器的 oracle；带静态成本函数与单节点电路工厂的 `onnx.helper` 建图库；三级验证器（官方样例 + N 个现场生成的新样例 + 约束审计，均在隔离子进程中运行）；以及自动拟合器（单 Conv 整数权重 LP、两层 torch）。

**发现**      **cost model** 把评分器逆向成"物理定律"：cost = 参数 + 中间张量字节；输入/输出张量与节点属性免费（单节点电路  $\approx 0$  内存）；log 尺度意味着重设计  $\gg$  微调（成本砍半仅 +0.69）。这决定了每一个设计选择。

**anti-overfit** 私有集由生成器采样，故拟合与验证都用  $\geq 3000$  现场生成的新样例 —— 官方约 265 样例会假阳性。

**key lesson** 一个 LP 拟合的 conv 本地过 3000 新样例，Kaggle 却得 0 分：LP 找到的是"能分开当前样本的权重"，不是真规则。结论 —— 拟合类电路对私有集不可信；只有手写、实现真规则、双侧 margin  $\geq 1$  的整数权重电路才稳健。

**audit** 逆向审计评分器有无可利用漏洞（早期  $>9000$  的漏洞分已被主办方重评打回），确认榜首  $\sim 8000$  是合法 golfing，无残余漏洞。

**结论**      最强公开基线（LB 7243）是社区三个月精调的强 bundle，在很多题上已贴近物理成本下界；现实合法天花板约 7300–7500，前 10 ( $\sim 7920$ ) 需要数百题深度 golfing。边际增益只来自基线未覆盖、需要真实空间计算（enclosure / 射线投射 / 计数）的结构，难以廉价电路化。

**教训**

- 遇到"few-shot 学习"外壳的比赛，先查是否存在生成器/参考实现能把学习退化成编译 —— 最大的杠杆。
- 把任何"最小化"比赛的评分函数当物理定律：实测每种算子/dtype/结构的确切成本，画出设计阶梯，据此选型 —— 绝不凭直觉。

- 区分"拟合出通过样本的函数"与"实现产生样本的真规则"。有 oracle 时永远实现真规则——这是唯一对私有分布稳健的东西。

状态

相对强基线的 Kaggle 净增益很小 (task192 +1.0、task243 +0.19、若干题稳健化后净~0)。两个 ORT 平台 bug 耗费了实打实的时间 (group-Conv 平铺；同进程 session 污染——用子进程评测解决)。int8/QLinearConv 两层路线 (约 10 个非线性可分题的正解) 已设计未完成。持久的产物是那份可迁移的技术总结，为 ARC-AGI-2 准备。

ONNX · onnxruntime

program synthesis

scorer reverse-engineering

integer-weight circuits

LP / torch fitting

subprocess batch eval

03

Nemotron Post-training Pipeline

RESEARCH

Python · A800 80GB · 2026-06-10 → 06-15

面向英伟达 **Nemotron-3-Nano-30B**（MoE + Mamba-Transformer 混合）推理模型的可复现后训练流水线。LoRA 微调失败；最有价值的产物是一次**被彻底归因的失败**——诊断出根因、六份事后分析、以及设计好的恢复路线。

**BASELINE** rule-based solver EM 0.7516 (100% format)    **INFRA** 30B MoE in 4-bit on one GPU (40 GB)

**OUTCOME** SFT loss 0.33, generation collapsed to EM 0%

|       |                                                                                                                                                                                                                                                                                                               |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 目标    | 在单卡云 GPU 上，通过"规则 teacher → CoT-SFT → 偏好学习"的流水线，把一个 30B 推理模型后训练成能产出符合比赛格式的 boxed CoT 答案。                                                                                                                                                                                                                       |
| 做了什么  | <ul style="list-style-type: none"><li>确定性规则求解器作为 teacher（EM 0.7516、格式 100%）—— 用干净蒸馏标签替代有噪声的模型输出。</li><li>单卡 4-bit 加载 30B MoE；发现并 monkey-patch 了 Nemotron-H 集成中的 <code>cache_position=None</code> bug；实现只对回复 token 计损失的 masked SFT collator。</li><li>多维奖励塑形（正确性 + 格式 + 推理 + 长度），并为 GRPO/DPO 自动构建偏好对。</li></ul> |
| 失败与诊断 | SFT loss 收敛到 0.33，但自回归生成崩溃成重复文本（EM 0%）。根因：Unsloth 与混合架构路径上 teacher-forcing 与生成的不一致。NeMo-RL 集成因环境复杂度（CUDA13 / mcore）放弃。                                                                                                                                                                                        |
| 教训    | 六份事后分析；设计好的恢复路线（native PEFT 蒸馏 + 强制推理前缀 + 用后训练的 teacher）。带走的永久流程规则：任何 500 步正式训练之前，先用 5 条样本 / 50 步跑通完整"训练→生成"回路。                                                                                                                                                                                               |

Transformers · PEFT · Unsloth · TRL

4-bit quantization

masked SFT collator

GRPO reward shaping

upstream bug patch

面向某 Kaggle 模拟赛的结构化多智能体博弈程序。真正的内容不是 agent 本身，而是**评估方法论**：与官方解释器逐函数一致的世界模型、双边自对弈、风格多样的对手池，以及对"离线指标为何无法预测线上排名"的诚实研究。诚实定性为**严格评估下的启发式搜索，而非深度强化学习**。

|      |                                                                                                                                                                                                               |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 方法   | <ul style="list-style-type: none"><li>世界模型从官方解释器逐函数移植，单元测试验证（10 回合旋转预测误差 &lt; 0.5 单位）—— 在真实物理上规划，而非近似。</li><li>多数参赛者跳过的评估装置：双边对调消除位置偏差、配置扰动生成四种对手风格避免对单一镜像过拟合、全轮循环 Elo 竞技场。</li><li>在对手池上做坐标下降参数搜索。</li></ul> |
| 关键发现 | <ul style="list-style-type: none"><li>参数交互：某参数单独调 -56 Elo，联合搜索 +108 —— 单参数消融会误导。</li><li>离线/线上效率落差：离线 +108 Elo 却对应线上排名下降。我记录了这一落差，并把每个被否决的实验连同 Elo 变化保留在代码库里，而非只报好看的数字。</li></ul>                             |
| 结论   | 参数搜索在中游平台期见顶；突破需要新能力（前瞻 / 同步走子搜索），而非继续调参。可迁移的教训关于评估效率：一个离线装置可以内部严谨，却仍然无法预测真实目标。                                                                                                                               |

- interpreter-faithful physics
- unit-tested world model
- bilateral self-play Elo
- 4-style opponent pool
- coordinate-descent search

一个跨设备任务系统：在手机上提交自然语言任务；路由引擎根据隐私、能力与负载决定在哪台设备（手机/桌面/云）执行；执行过程带 checkpoint 跨设备接续；每次状态变更都是不可变事件。建立在四条原则上—— **Task-first**、**Schema-first**、**控制面优先**、**本地优先**。

PROTOCOL 42 JSON Schemas · 7 protocol docs + ADRs   SURFACE 5 codebases (TS/Py/Swift/Kotlin)

INFERENCE LiteRT-LM on Metal GPU, 4–6s/task

|      |                                                                                                                                                                                                                                                                                                                                                                            |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 架构   | TypeScript/Node 控制面拥有全部状态（SQLite、Repository 模式、每个边界都有 AJV 校验、7 组 API 路由）。Python asyncio 桌面运行时执行工具。iOS（SwiftUI）、Android（Compose/Hilt）、macOS（菜单栏）原生宿主通过 Bonjour 互相发现。端侧推理基于 LiteRT-LM 的 Metal GPU 调度（单引擎、按任务开会话）。                                                                                                                                                            |
| 数据模型 | 三层对象模型：Device → Task → Run，Event 是唯一的状态变更入口。所有类型从 42 份 JSON Schema（Draft 2020-12）生成——四种语言之间零手写 DTO。HandoffPayload 携带 checkpoint + 工具历史 + 上下文并带幂等键，使手机上开始的任务能在 Mac 上完成。                                                                                                                                                                                                     |
| 核心机制 | <ul style="list-style-type: none"><li>事件溯源——Task/Run 状态只能经 <code>EventService.processStateTransition</code> 变更；任何路由处理器不得直接改状态，从而得到完整、可回放的审计历史。</li><li>路由引擎——隐私边界（<code>local_only</code> / <code>private_lan</code> / <code>cloud_ok</code>）做硬过滤、设备状态做软排序；同一任务可合法地跑在三处，选择通过事件流解释。</li><li>Fail-closed 安全（作为专门的加固阶段加入）——中央安全策略、灾难性命令模式筛查、人在回路审批门；审批 UI 不可达时拒绝执行。</li></ul> |
| 设计取舍 | 协议优先，因为一份规范驱动四种语言能为独立开发者消除大部分联调摩擦；而一个会碰你文件/日历的助手必须可审计、失败即关闭、默认数据不出本地。一份 ADR 记录了为何选 JSON Schema 而非 Zod / Protobuf / OpenAPI（多语言 + LLM 可读 + 类型生成工具链成熟）。                                                                                                                                                                                                                      |
| 状态   | 约 1 个月内完成端到端的“双运行时”生产形态系统（手机 + 桌面）；云端运行时与 Web 控制台有规范但为占位。Android 端比 iOS 薄；TS 单测覆盖落后于 Python 侧（16 个测试）；事件推送为轮询（WebSocket 排 v2）。                                                                                                                                                                                                                                             |



一个 macOS 菜单栏工具：按住 `Fn` 说话、松开 —— 实时语音被转写并注入任何文本框。纯 **Swift**，零第三方依赖，通过延迟工程让听写"即说即得"。

|                                                                                                                                                                  |                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 核心机制                                                                                                                                                             | <ul style="list-style-type: none"><li>通过 CGEventTap 全局监听 <code>Fn</code>，并以 NSEvent 降级保证系统版本兼容；处理 Accessibility + 麦克风 + AppleEvents 权限。</li><li>AVAudioEngine 采集，Float32 → PCM-16 线性插值重采样（44.1/48k → 16k）；通过实现 OpenAI-Realtime 兼容协议的 WebSocket 流式传输（DashScope Qwen3 ASR）。</li></ul> |
| 设计决策                                                                                                                                                             | <ul style="list-style-type: none"><li>预录制缓冲：<code>Fn</code> 按下瞬间即开始录音并缓存，socket 连接期间缓存、连上后冲刷 —— 把感知延迟压到远低于典型连接耗时。</li><li>中文输入法无缝处理：检测到中文输入法时切到 ABC 注入、完成后恢复 —— 解决 macOS 上一个默认无解的真实痛点。支持 7 种语言。</li></ul>                                                                           |
| 状态                                                                                                                                                               | 可用（Makefile 一条命令构建，附诊断脚本）。云端 ASR，需要网络与付费 API key；仅 macOS；项目较新，尚无长期用户数据。                                                                                                                                                                                                             |
| <div>Pure Swift · zero deps</div> <div>CGEventTap + NSEvent</div> <div>AVAudioEngine resampling</div> <div>WebSocket streaming ASR</div> <div>menu-bar app</div> |                                                                                                                                                                                                                                                                                     |

一个急诊分诊决策栈。比赛中途我发现 **99.7% 的主诉短语单调映射到唯一分诊等级** —— 数据集奖励的是查表而非建模。我记录了泄漏、拒绝利用它，转而构建"文本盲"的生理学模型，并包上校准、conformal 不确定性、公平性审计和运营控制台。用时三天。

**MODEL** text-blind multi-task GBDT, QWK 0.9287    **CALIBRATION** ECE -46% (isotonic)

**TESTS** 12/12 (incl. leakage guards)

|       |                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 数据与特征 | 患者级防泄漏四表联接；以生理学为根基的特征（NEWS2、休克指数、GCS、缺失指示器），训练与在线推理共用同一个 <code>transform()</code> —— 从构造上杜绝 train/serve 偏移。                                                                                                                                                                                                                                                                                               |
| 决策栈   | <ul style="list-style-type: none"><li>模型 —— 多任务 GBDT（分诊等级 + 住院 + 停留时长），QWK 0.9287；阈值调优提升 acuity-4 召回而不损危重召回。</li><li>治理 —— isotonic 校准（ECE -46%）；分布自由的 conformal 集合将约 89% 病例推给人工复核（这本身就是标签在生理学上欠定的证据）；按熵排序的复核队列。</li><li>公平性 —— bootstrap 95% 置信区间（而非点估计）揭示爱沙尼亚语人群 3 倍欠分诊差距、疼痛评分缺失时 QWK 崩到 0.78，并标记为数据质量红旗而非调参掩盖。</li><li>运营 —— 医生可读的优先级方程（透明加权项，非黑盒）、资源桶、急诊负载预测，以及带人工覆盖日志的四视图仪表板（队列/患者/运营/审计）。</li></ul> |
| 判断    | 靠主诉查表训练的分诊系统放进真实急诊室是危险的；医院需要的是可信的不确定性 —— 哪约 10% 可安全自动分诊（该切片欠分诊率仅 0.16%）、哪些必须交给护士。我还试过一条"生理学安全网"规则，结果降低了 QWK，却照样报告 —— 因为那是关于标签的证据，而非需要藏起来的失败。                                                                                                                                                                                                                                                              |
| 局限    | 合成比赛数据 —— 无前瞻或多中心验证（TRL 3-4，明确不是医疗器械）。公平性差距被量化但未解决；acuity-4 仍是最难类别；优先级权重为手工调。                                                                                                                                                                                                                                                                                                                             |

LightGBM · multi-task

isotonic calibration

conformal prediction (APS)

bootstrap fairness CIs

shared transform()

4-view dashboard

面向某 Kaggle 竞赛的分层多单位策略 agent —— 解析 → 世界状态 → BFS 规划 → 角色策略 → 冲突解算  
—— 一天内完成构建、评估并纳入 CI。

- 架构
- 单一职责的分层模块；每个单位提出行动，三轮迭代解算器检测冲突、让高优先级胜出、为落选者重规划 —— 从构造上保证合法性。
  - 77 个策略阈值全部集中在一个配置模块 —— 整套策略可调而无需碰逻辑；按回合数与地图压力驱动开局/经济/生存三阶段切换。
  - 跨回合记忆瓦片、矿点与敌方轨迹，减少雾战下的重复探索。

结果

16 种子评估：平均奖励 168.5，随机基线为 0。附 Makefile、批量评估脚本与 GitHub Actions CI。诚实地说是 v1 启发式基线 —— 参数靠经验调优而非系统搜索；测试较轻量。

BFS planning under fog

3-round conflict resolution

77 centralized params

cross-turn memory

Makefile + CI

面向竖屏短剧的端到端 AI 制片流水线：输入一句话，输出带字幕的 9:16 视频 —— 剧本 → 分镜 → 逐镜头视频生成 → 视觉模型评审每个镜头并驱动定向重试 → 多语言字幕。

定位

面向短剧工作室（TikTok / Reels / 小红书）的虚拟制片工作流：把"剧本到成片"周期从数天压到数小时、保持角色跨镜头一致、并从一份剧本本地化到多语言。

- 如何构建
- 闭环评审而非一次性流水线：每个片段抽 3 帧，由 VL 模型按叙事契合、角色一致性、技术质量打分；不合格生成定向修复提示词（如"脸部模糊 → 加清晰度关键词"），在重试预算内进行。
  - 跨镜头角色一致性：规范参考肖像锁入角色库，作为图生视频首帧注入（一致性实测 9-10）。
  - 每个阶段都是可回放的 Pydantic JSON 产物（故事圣经 → 镜头规格 → QA 报告 → 剪辑表）；在大纲/分镜/成片设人工审批门；每次运行有成本上限。

状态

产出 6 部端到端成片（含一部 26 秒短剧）。带实时事件流的 FastAPI 仪表板。TTS/口型同步已设计未实现；黑客松最后一轮撞上 API 配额限制；云部署脚本已写好但未实地验证。

FastAPI · asyncio

Pydantic JSON artifacts

Qwen-Max planner

Qwen-VL critic

I2V/T2V generation

ffmpeg · multi-lang SRT

一个完全在浏览器内运行的法语学习助手 —— LLM 词典、双语语音输入、拍照 OCR、真人朗读 —— 无服务器、无 API、无需网络。

定位

隐私优先的离线语言学习："模型即应用"。学习者一次性选择模型档位，然后查词、听写或拍下菜单 —— 数据永不离开设备。

- 如何构建
- WebLLM 运行量化 Qwen（0.5B/1.5B/3B 档，350MB–1.7GB）；不到 2KB 的系统提示词加 JSON schema 约束输出，把小的本地模型变成结构化词典（音标、义项、变位、词源、名言），解析失败时优雅降级为自由文本。
  - 三种输入模态汇聚到同一推理路径：键入文本、长按双语语音（WebGPU 上的 Whisper-tiny）、菜单路牌拍照 OCR（Tesseract.js）；朗读用 Web Speech API。
  - 围绕 GB 级权重设计的 Service Worker 缓存策略：应用代码网络优先、不可变 CDN 模型分片缓存优先，并让推理引擎自管权重缓存以避免重复占用数 GB。

状态

生产级 PWA（可安装、离线可用）。3B 档词条偶有瑕疵（重要查询建议交叉核对）；WebGPU 浏览器覆盖仍在演进；首次下载需在 Wi-Fi 下耐心等待。

WebLLM · quantized Qwen

Whisper-tiny · WebGPU

Tesseract.js OCR

Web Speech TTS

PWA · Service Worker

vanilla JS

一个已上线的工具，把艺术家的作品变成精致的 Instagram 内容 —— 单帖、系列合集、视频故事 —— 由一套原生 JS 代码同时作为网站、PWA 和 iOS 应用运行。现已在线。

定位

面向艺术家与摄影师的、隐私优先且无后端的发布工作室：上传 → 调整 → 排版 → 导出 → 分享，全程本地，不追踪、不上传。

如何构建

- 手写的 Canvas 2D 引擎：三角网格变形实现透视（梯形）校正、五层图像调整、视频幻灯片的 Ken Burns 运镜、按 Instagram 上传顺序导出的 3x3 壁画分割。
- IndexedDB 本地存储；一套代码经 Capacitor 覆盖 Web、PWA 与 iOS（WebView），无平台特定 hack；8 种界面语言。
- 三个工作台 —— 单幅 / 系列 / 故事。我还做过 AI 文案功能，评估后主动移除 —— 知道砍什么也是产品判断力的一部分。

状态

已部署在 GitHub Pages (artworktopost)；27 天 29 次提交；iOS 构建完成并备齐 App Store 素材。单文件代码库（约 1.6k 行 JS）接近可维护性边界；无自动化测试；付费墙框架已搭但未启用。

vanilla JS · single file

Canvas 2D engine

IndexedDB · local-first

Capacitor iOS

GitHub Pages · live

职业经历

金融市场与对公银行，及产品与增长。

BNP Paribas — 对公银行部实习生

巴黎 · 2025.03 – 2025.08

- 与客户覆盖、产品、法务及风险团队协作，为国际客户设计量身定制的金融解决方案，聚焦交易型服务（现金管理、外汇对冲、短期融资），在紧张周期内与前台销售及交易员一同交付有竞争力的报价。
- 为高价值客户制作并演示投行推介材料（pitchbook），结合市场趋势分析、定价情景与跨境流动性方案；参与面向全球企业客户的商业策略与产品落位。
- 协助复杂交易的执行：进行国别风险分析（尤其新兴市场）、参与法律与授信条款谈判、优化风险缓释工具（担保、供应链金融）。
- 与客户经理（RM）紧密协作，识别并推介交叉销售机会（外汇、投资方案、结构性现金产品），并针对 TMT 与电子行业进行季度客户分析。
- 通过自动化报告看板与监控关键客户资金流支持前台运营效率，并与中后台保持常态沟通。
- 协助推动 AI 工具在行内落地 —— 在 Open WebUI 与 Dify 上搭建 RAG 知识库以提升客户答复效率 —— 服务客户包括宁德时代、比亚迪、支付宝、美的、TCL、安踏、京东等。

---

**HSBC — 全球市场部 流动性与投资实习生**

巴黎 · 2024.06 – 2024.11

- **资金 / 宏观 / 利率分析** —— 跟踪央行利率（美联储/欧洲央行/英格兰银行）与货币市场以优化 EMEA 短期组合；与资金部协作，在 LCR/NSFR 约束下进行流动性配置与曲线布局。
- **定价与建模** —— 为批发存款开发动态定价模型（基准校准、内部资金曲线整合）；与交易台协作。
- **组合优化** —— 结合宏观信号与流动性约束优化 EMEA 短期利率组合；构建 VBA/Excel 自动化工具实时跟踪利率变动与定价缺口以识别套利机会。
- **治理与报告** —— 负责货币市场统计报告；参与 ALCM 委员会（缺口风险分析、融资成本优化、敏感头寸跟踪）；用 VBA 自动化报告。

---

**TikTok — 增长策略实习生**

北京 · 2023.02 – 2023.05

- 参与 10+ 战略项目，通过市场趋势与竞争格局分析，使 SaaS/AI 业务市场份额提升 23%。
- 用 SQL 与 Excel 分析财务与运营数据，为 20+ 人团队产出周报，支持最优资源配置并提升 ROI。
- **主导流程优化** —— 与设计、产品、IT 团队合作开辟新线上渠道，一个月内将用户转化率提升 13%。
- 统筹跨职能项目管理，交付与业务目标一致的战略举措。

---

**Uber China · DiDi — 产品管理与用户运营实习生**

北京 · 2022.08 – 2023.01

- **业务运营优化** —— 负责 3 个优化运营的项目，运用建模与数据分析为用户互动与降本制定具体策略。
- **数据与流程分析** —— 用 Python、SQL、VBA 分析大规模数据集并优化流程，识别优化机会、构建预测模型、落地数据驱动的用户互动策略。

---

**教育背景**

---

**ESCP Business School（伦敦、巴黎校区）**

2023 – 2026

管理学硕士（金融专业）。资本市场、金融建模、公司估值、衍生品、结构性融资。

---

**Beijing International Studies University 北京第二外国语学院**

语言与文化学士（法语、日语）；中国传媒大学网络与新媒体辅修；自学心理学并获耶鲁大学 Coursera 证书。

2019 – 2023